

LPR381 — Linear Programming

Programming Project — Group Plan

Language: C# (.NET) | Tool: Visual Studio | Output: solve.exe

1. Project Overview

The goal of this project is to build a menu-driven console application (solve.exe) using C# and Visual Studio that can:

- Read a Linear Programming (LP) or Integer Programming (IP) model from a plain-text input file.
- Solve the model using one of several algorithms selected by the user.
- Export all results — canonical form and every tableau / iteration — to an output text file.
- Perform sensitivity analysis on the solved model.
- Detect and handle special cases such as infeasible and unbounded models.

The example problem used for demonstration (as specified in the project brief) is the binary Knapsack IP:

```
max  +2 +3 +3 +5 +2 +4
+11 +8 +6 +14 +10 +10  <=  40
bin bin bin bin bin bin
```

The Santa's Workshop LP (provided in the Excel reference file) will be used to verify the Primal Simplex and sensitivity analysis outputs.

2. Mark Allocation

Criteria	Marks
Input / Output / Interface / Error Handling	17
Primal Simplex Algorithm	4
Revised Primal Simplex Algorithm	4
Branch & Bound Simplex Algorithm	20
Branch & Bound Knapsack Algorithm	16
Cutting Plane Algorithm	14
Sensitivity Analysis	25
TOTAL	100

3. Member Allocation Summary

Member	Primary Responsibilities	Marks	Algorithms / Modules
Member 1	Project Lead & Core Infrastructure	~21	Primal Simplex, Input/Output/Interface/Error Handling
Member 2	Revised Simplex & B&B Simplex	~24	Revised Primal Simplex, Branch & Bound Simplex

Member	Primary Responsibilities	Marks	Algorithms / Modules
Member 3	Knapsack & Cutting Plane	~30	B&B Knapsack, Cutting Plane
Member 4	Sensitivity Analysis	~25	All SA operations, Duality

4. Detailed Member Responsibilities

Member 1 — Project Lead & Core Infrastructure (~21 marks)

4.1 Project Architecture

- Create the Visual Studio solution with a clean folder structure:
 - Models/ — LPModel, Tableau, Node, SensitivityResult classes
 - Parsing/ — InputFileParser
 - Output/ — OutputFileWriter
 - Algorithms/ — one file per algorithm
 - SensitivityAnalysis/ — all SA operations
 - Utils/ — shared helpers (rounding, matrix operations)
- Design all shared data structures that every other member's code will consume. Agree on these in Week 1 before anyone else starts coding.

4.2 Input File Parser (3 marks)

The parser reads the plain-text input file. The format is:

- Line 1: max/min keyword, then sign operators and coefficients for each decision variable's objective function coefficient.
- One line per constraint: sign operators and technological coefficients, a relation ($\leq$, $\geq$, $=$), and the RHS value.
- Final line: sign restrictions ('+', '-', 'urs', 'int', 'bin') for each variable.

The parser must handle any number of decision variables and any number of constraints (random amounts as per minimum requirements).

4.3 Output File Formatter (2 marks)

- Write all results to a text file: canonical form header, then every tableau iteration produced by whichever algorithm ran.
- All decimal values rounded to exactly 3 decimal places.
- Output should mirror what is displayed in the console so the file is a complete record.

4.4 Menu System & Interface (5 marks)

- Main menu: load input file, select algorithm, run sensitivity analysis, exit.
- Algorithm sub-menu: Primal Simplex, Revised Primal Simplex, B&B Simplex, Cutting Plane, B&B Knapsack.
- Sensitivity analysis sub-menu: all 12 listed operations plus duality.
- All menus are numbered and navigated by keyboard input. No GUI required.

4.5 Canonical Form Builder

- Convert the parsed model into canonical form (add slack/surplus variables, rearrange objective row).
- Display the canonical form on screen and write it to the output file before running any algorithm.

4.6 Primal Simplex Algorithm (4 marks)

- Display canonical form first, then execute the Primal Simplex method.

- Display every tableau iteration (T-1, T-2, ... T-n*) including the θ column for ratio test.
- Identify and display the pivot element, entering variable, and leaving variable at each step.
- Terminate when all objective row coefficients are non-negative (maximisation) and display the optimal solution.
- Use the Santa's Workshop LP as the test case — expected optimal: $x_1=20$, $x_2=60$, $z=180$.

4.7 Error Handling (5 marks)

- Detect infeasible models: all ratios negative during the ratio test → report infeasibility and stop.
- Detect unbounded models: no positive entry in the pivot column → report unboundedness and stop.
- Validate input file format and provide a clear error message if the file is malformed.
- Wrap algorithm entry points in try/catch and surface friendly error messages rather than stack traces.

4.8 Integration (ongoing)

- In Week 5, pull each member's algorithm module into the solution and verify the menu routes to each one correctly.
- Run the Knapsack IP and Santa's Workshop LP through every algorithm and confirm outputs match expected values.

Member 2 — Revised Simplex & Branch and Bound Simplex (~24 marks)

4.9 Revised Primal Simplex Algorithm (4 marks)

The Revised method does not recompute the full tableau each iteration — instead it maintains the basis inverse B^{-1} and updates it using the Product Form of the Inverse (eta vectors). Refer to the Excel reference file (sheet 'Algo - Revised Primal Simplex') for the exact layout expected.

- For each iteration, display:
 - Current basic variable set (X_{bv}) and non-basic variable set (X_{nbv})
 - Current basis B and its inverse B^{-1} (or eta factor matrix E)
 - C_{bv} vector and $C_{bv}B^{-1}$ (shadow price row)
 - Price Out: compute $C_j^* = C_{bv}B^{-1}A_j - C_j$ for each non-basic column; select most negative as entering variable
 - Product Form update: compute $A^* = B^{-1}A$ for the entering column, perform ratio test (θ), identify leaving variable
 - Update B^{-1} using the eta matrix
- Terminate when all Price Out values ≥ 0 and display the optimal solution.

4.10 Branch & Bound Simplex Algorithm (20 marks)

This is the highest-weighted single algorithm. Work methodically: get the LP relaxation solving first, then layer in branching and fathoming.

- Step 1 — LP Relaxation: solve the relaxed LP using Primal Simplex (you can call Member 1's Primal Simplex method).
- Step 2 — Branch: if the relaxation solution has a fractional variable, create two sub-problems — one adding a $\leq$ floor constraint and one adding a $\geq$ ceil constraint on that variable.
- Step 3 — Node management:
 - Maintain a candidate list of open nodes (sub-problems not yet solved or fathomed).
 - At each step, select the best candidate (highest upper bound for maximisation).
- Step 4 — Fathoming rules — a node is fathomed (pruned) when:
 - Its LP relaxation is infeasible.
 - Its LP relaxation upper bound $\leq$ current best integer solution (bound fathom).
 - Its LP relaxation solution is fully integer (integrality fathom) → update best solution if better.

- Step 5 — Backtracking: when all children of a node are fathomed, backtrack to the next open node in the candidate list.
- Display all tableau iterations for every sub-problem node.
- At termination, display the best candidate node and the optimal integer solution.

Important: the program should create and explore all possible sub-problems before declaring the optimal solution. Do not stop at the first integer solution found.

Member 3 — Knapsack & Cutting Plane (~30 marks)

4.11 Branch & Bound Knapsack Algorithm (16 marks)

This is a specialised B&B designed for binary knapsack problems. It exploits the knapsack structure for more efficient bounding.

- Sort items by profit/weight ratio in descending order.
- Upper bound at each node: use the LP relaxation of the remaining items (fractional knapsack bound).
- Tree traversal: explore left child (item included, $x=1$) before right child (item excluded, $x=0$) — greedy first.
- Backtrack when the upper bound of a node $\leq$ current best solution value.
- Fathom nodes that exceed the knapsack capacity or whose bound is not better than the current best.
- Display all sub-problem nodes explored, the bound at each node, and the items selected.
- At termination, display the best candidate and the optimal solution.

Test case: max $+2 +3 +3 +5 +2 +4$ subject to $+11 +8 +6 +14 +10 +10 \leq 40$, all binary. Expected optimal: select items 2, 3, 4 (0-indexed) with $z=10$.

4.12 Cutting Plane Algorithm (14 marks)

Also called the Gomory Cutting Plane method. It iteratively adds valid cuts to the LP relaxation until the LP optimal solution is integer.

- Start with the LP relaxation. Solve it using the Revised Primal Simplex (call Member 2's method or use your own).
- For each iteration, display Product Form and Price Out steps.
- If the optimal solution is not fully integer, identify a fractional basic variable.
- Generate a Gomory cut from the fractional row of the optimal tableau and add it to the model as a new constraint (with a new slack variable).
- Re-solve the extended model. Repeat until all variables are integer or infeasibility is detected.
- Display every full tableau after each cut is added.
- Display the final integer optimal solution.

Note: because the Cutting Plane method can cycle on degenerate problems, add a maximum iteration limit (e.g. 50 cuts) and report if reached.

Member 4 — Sensitivity Analysis (~25 marks)

All sensitivity analysis operations work on the optimal tableau produced by whichever algorithm was used to solve the model. Make sure Member 1 exposes the final optimal tableau through the shared data structures before starting implementation.

4.13 Objective Function Coefficient Ranges

- Non-Basic Variable range: for a chosen NBV coefficient c_j , compute how much it can increase/decrease before the current basis changes. Display the allowable range.
- Non-Basic Variable change: apply a user-specified change Δ to a chosen NBV coefficient, update the objective row, and re-display the (still-optimal) solution or flag that optimality is lost.
- Basic Variable range: compute the range for a chosen BV coefficient c_{bv} using the $C_{bv}B^{-1}$ ratios. Display the allowable range.

- Basic Variable change: apply a user-specified change Δ to a BV coefficient, recompute $CbvB^{-1}$, and verify/restore optimality.

4.14 Constraint RHS Ranges

- RHS range: for a chosen constraint's RHS value b_i , compute how much it can change while keeping the current basis feasible (using $B^{-1}b$ ratios). Display the allowable range.
- RHS change: apply a user-specified change Δ to a chosen b_i , recompute $B^{-1}b$, update decision variable values, and display the new solution.

4.15 Non-Basic Variable Column Ranges

- Column range: for a chosen variable's column in the NBV matrix, compute the range of a single coefficient a_{ij} that keeps the current basis optimal.
- Column change: apply a user-specified change Δ to a_{ij} , recompute the affected Price Out, and display whether optimality is maintained.

4.16 Structural Changes

- Add new activity: accept a new decision variable (its objective coefficient and constraint column), compute its reduced cost, and determine if it should enter the basis.
- Add new constraint: accept a new constraint row, add it to the optimal tableau, and use dual simplex or re-solve to restore feasibility/optimality.

4.17 Shadow Prices

- Display the shadow price (dual variable) for each constraint: $y_i = CbvB^{-1}$ for each row.
- Refer to the Excel reference file (sheet 'LP') for the expected shadow price values on the Santa's Workshop problem.

4.18 Duality

- Apply duality: construct the dual LP from the primal model and display both models side-by-side.
- Solve the dual model: run Primal Simplex on the dual and display its optimal solution.
- Verify duality type: if $\max z$ (primal) = $\min w$ (dual), the models have Strong Duality — confirm this and display the duality gap (should be 0). Otherwise report Weak Duality.

For the Santa's Workshop LP, the expected result is $\max z = \min w = 180$ (Strong Duality). Use this as your verification target.

5. Coding Standards & Shared Conventions

5.1 Language & Tooling

- Language: C# (.NET Framework or .NET 6+). Use whatever version your Visual Studio supports.
- Every file must use the same target framework — agree on this in Week 1.
- All code must compile without warnings in Release mode.

5.2 Naming & Comments

- Classes: PascalCase (e.g. LPModel, BranchAndBoundNode)
- Methods: PascalCase (e.g. SolvePrimalSimplex())
- Variables: camelCase (e.g. pivotColumn, enteringVariable)
- Every class and public method must have an XML doc comment (`/// <summary>...</summary>`).
- Add inline comments for any non-obvious logic, especially pivot operations and fathoming conditions.

5.3 Data Structures (set by Member 1 in Week 1)

- LPModel: `int NumVars, int NumConstraints, double[] ObjectiveCoefficients, double[,] ConstraintMatrix, double[] RHS, string[] SignRestrictions, bool IsMaximisation`
- Tableau: `double[,] Data, string[] ColumnLabels, string[] RowLabels, int NumRows, int NumCols`

- BranchNode: int Id, int ParentId, LPModel SubProblem, Tableau OptimalTableau, double Bound, bool Fathomed, string FathomReason

5.4 Rounding

- All values displayed or written to file must be rounded to 3 decimal places using `Math.Round(value, 3, MidpointRounding.AwayFromZero)`.

5.5 Output Format

- Every algorithm must print: the algorithm name, the canonical form, then each iteration labelled T-1, T-2, ... T-n*, then the optimal solution.
- For B&B algorithms: label each node clearly (Node 1, Node 2, etc.) and show the bound.
- The same output must be written to the output text file.

5.6 Error Handling

- Never let an unhandled exception crash the program.
- Catch `ArgumentException` and `InvalidOperationException` at algorithm boundaries.
- Report infeasible / unbounded with a clear message and return to the main menu.

6. Testing Plan

Each member is responsible for unit-testing their own module. The following test cases must all pass before the final integration in Week 5:

- Santa's Workshop LP (from Excel reference file): Primal Simplex must produce $x_1=20$, $x_2=60$, $z=180$ in 4 tableau iterations.
- Santa's Workshop LP: Revised Primal Simplex must produce the same optimal solution using Product Form and Price Out steps.
- Knapsack IP (max $+2+3+3+5+2+4$, constraint ≤ 40 , all binary): B&B Knapsack must find the optimal integer solution.
- Knapsack IP: B&B Simplex must find the same optimal solution via LP relaxation and branching.
- Knapsack IP: Cutting Plane must find the same optimal integer solution by iteratively adding Gomory cuts.
- Sensitivity Analysis: shadow prices from Member 4 must match the reference Excel file values for Santa's Workshop.
- Error case: test with an infeasible model and confirm the program reports the special case and does not crash.
- Error case: test with an unbounded model and confirm the program reports it correctly.

7. Deliverables & Submission Checklist

- Visual Studio solution (.sln) with all source code.
- Compiled solve.exe that runs from the command line.
- Sample input text file (the Knapsack IP) for the video demonstration.
- Video demonstrating every criterion in the mark allocation table — cover all algorithms, all SA operations, error handling, and interface navigation.
- PDF with a OneDrive link if the video file is too large to upload directly.
- (Optional) PDF listing individual contributions if any member did not contribute equally — include evidence of attempts to involve them.

8. Practical Tips

- Do the assignment (theory) before starting the project — the brief says this explicitly. Understanding the manual calculations from the Excel reference file will make coding each algorithm much faster.

- Member 1: finalise the LPModel and Tableau class signatures by end of Week 1 and share them as a header/interface so Members 2–4 can code against them without waiting.
- Member 2: implement B&B Simplex by reusing the Primal Simplex code from Member 1 — just call it inside a loop with modified constraint sets.
- Member 3: the Cutting Plane method can also reuse Member 1's Primal Simplex or Member 2's Revised Simplex — decide which in Week 1.
- Member 4: build SA against the Santa's Workshop LP first (you have the expected answers in the Excel file). This makes debugging much faster.
- Use Git (GitHub / Azure DevOps) for version control. Commit after every working feature. Never push broken code to main — use branches.
- Record the video in one session after integration testing is complete. Walk through each criterion in the order of the mark allocation table so nothing is missed.

— End of Project Plan —